

DEVELOPER ARCHITECTURE & SPECIFICATION

kaguya-ai

会話・記憶・音声・生活感を、
1台のWindows PCに統合する。

初見の開発者が「何を作っているか」から「どのファイルを触れば
よいか」まで順に理解できるよう、現行実装をアーキテクチャ・処
理フロー・DB・API・UI・状態遷移へ分解した資料。

FastAPI

TypeScript

PostgreSQL

Gemini

Tailscale

AivisSpeech



この資料の読み方

CONCEPT → FLOW → DATA → API → UI → OPS

全体像から入り、会話の1ターンを追い、最後にDB・API・ファイル構成へ降りる。
各章は「役割」と「境界」を先に示し、実装詳細はその後に置く。

01-04

何を作っているか

コンセプト、機能、アーキテクチャ、端末構成。

05-12

どう動くか

テキスト/音声、Controller、Mind、Living、学習、状態遷移。

13-24

どう実装されるか

DB、API、ネットワーク、UI、技術スタック、運用、ソースマップ。

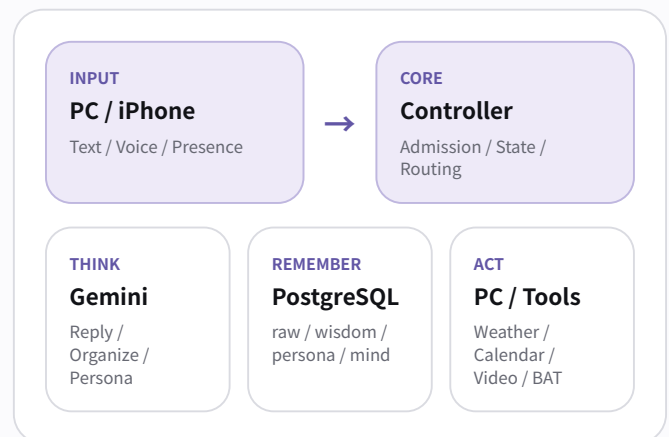
設計の中心

Controller が交通整理を行う。

UI、Gemini、記憶、Kaguya Mind、PC操作、定期処理を1つの会話状態へ束ねる。

1 active turn + 1 unsaved answer

同時生成を避け、保存失敗時に回答を再生成せず「保存のみ再試行」できる構成。



「使うAI」より、「一緒にいるAI」

PERSONAL / LOCAL-FIRST / LOW-FRICTION

Windows PCを母艦に、PC・家庭内Wi-Fi・本人のTailscale環境から同じ会話と記憶へアクセスする個人用AIキャラクター。

会話

Geminiによる文字チャット。
ストリーミング表示、会話履歴、参照記憶。

記憶

raw_memory → wisdom → persona。明示記憶と訂正・削除UIを備える。

音声

Gemini Live。字幕保存。
AivisSpeechまたはGemini音声で読み上げ。

生活感

Living Kaguya、Kaguya Mind、時間帯や利用傾向に応じた表情・行動。

生活支援

天気、予定、リマインダー、参照ファイル、設定変更を低コストで処理。

PC連携

登録済み動画再生とBAT実行。
任意パス/任意コマンドは受け付けない。

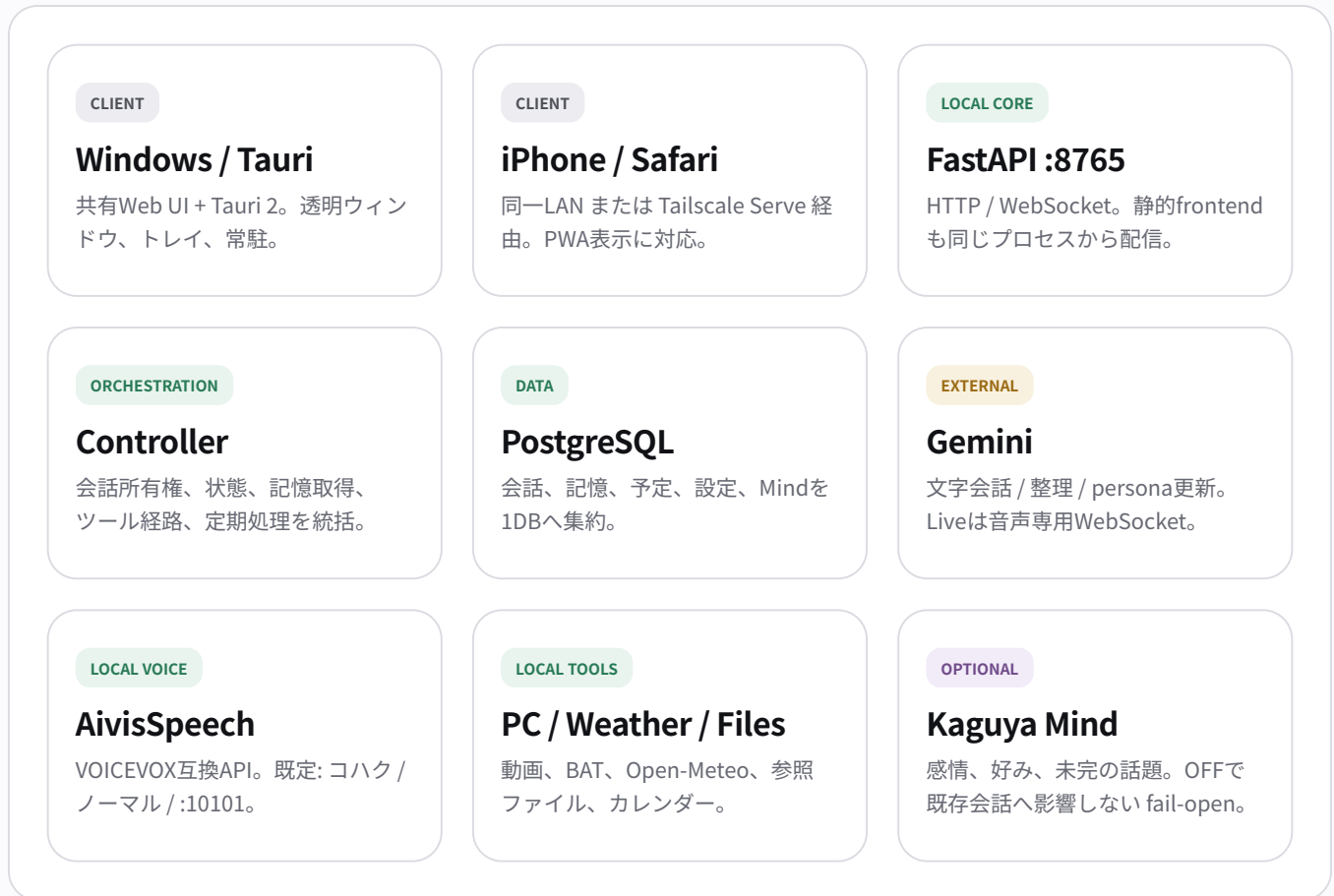


通常会話では不要なFunction Calling定義を送らず、天気などローカル確定できる処理はGeminiを経由しない。安定性と低トークン消費を優先。

● Core ● Local deterministic ● External AI / API ● Optional / experimental

PCを母艦にした、ローカル中心の構成

ONE PROCESS / ONE USER / SHARED WEB UI

**1**

FastAPI process / Uvicorn 1 worker

1

同時生成ターン

2

主要WebSocket: /ws, /voice

テキスト入力の1ターン

ADMIT → RECALL → GENERATE → SAVE

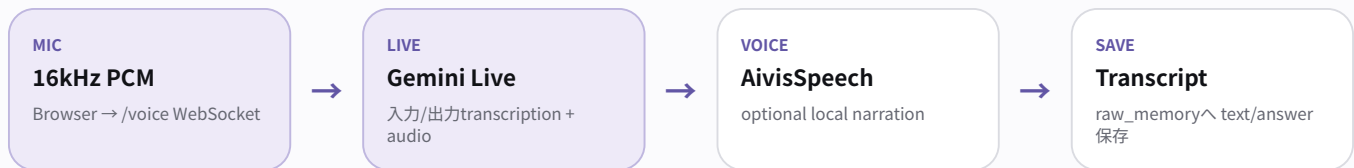
UI	1. chat.send turn_id(UUID), text, client_id, input_mode=text。送信端末はlocal draftを保持。
Controller	2. admission editing / active / unsavedを確認。activeがあれば busy。同一turn_idなら状態を再送。
Memory API	3. begin raw_memoryへuser発言をpendingで1回だけ保存。turn_id+roleの一意制約で重複防止。
Controller	4. local reactions Relationship feedback、簡易Mood、Kaguya Mind.before_reply、proactive activityを更新。
Tools	5. direct route first PC操作/天気/予定確認など、確定可能な処理はローカル経路へ。不要なLLM呼び出しを避ける。
Memory API	6. context + recall 直近10ターン、関連wisdom/persona、Relationship、必要ならMindを収集。
Gemini	7. reply 通常は低Thinking。Function Callingが不要ならstream。必要時のみ宣言を付与し、最大2件/1 round。
Controller	8. saving partial_answerを保持しphase=saving。cancelは保存中には受け付けない。
Memory API	9. complete assistant rowをcompletedで追加しuser rowをcompletedへ。保存失敗時はunsavedを保持。
UI	10. chat.completed answer / references / elapsed_ms / first_text_ms。全端末へbroadcast。

保存失敗は「再生成」しない

回答をメモリ上の unsaved に保持し、chat.retry_save で保存だけを再試行する。

音声は「Live会話」と「読み上げ」を分離

GEMINI LIVE + OPTIONAL LOCAL TTS



Session contract

項目	仕様
開始条件	active / unsaved / editing が空。session tokenとOriginを検証。
上限	最大600秒。入力待ち45秒。
入力	audio/pcm;rate=16000、1chunk ≤ 32768 bytes。
字幕	input/output transcriptionを逐次UIへ送信。
保存	音声そのものは保存せず、発話文字列と回答を会話履歴へ保存。
割り込み	NO_INTERRUPTION。interrupted通知時は未読TTSキューを破棄。

Local TTS path

AvisSpeech / コハク

VOICEVOX互換API。話者IDは固定値ではなく / speakers から名前+スタイルで解決。

24k

Hz / mono / 16-bit PCM

200

chars / TTS chunk max

- 最初の一言は短く区切り、発話開始遅延を抑える。
- 句読点単位で逐次合成し、受信と読み上げを別task化。
- ローカルTTS失敗時はGemini音声へフォールバック。

会話の所有権と副作用を1か所に集約

backend/app/controller.py

Admission

active / unsaved / editing / voice_active を判定し、二重生成と競合を防ぐ。

Progress

phase、partial_answer、first_text_msを管理し、0.1秒間隔でprogressをbroadcast。

Routing

PC action → direct tool → memory recall → Geminiの順に最短経路を選ぶ。

Persistence

begin / complete / fail をMemory APIへ依頼。曖昧な保存失敗で再生成しない。

Background

5秒tick。mood、proactive、reminders、memory jobsを軽量に駆動。

Cross-device

状態・表情・会話完了・設定変更を全WebSocketへ同期。

主要インメモリ状態

変数	意味
active	現在処理中の1 turn
unsaved	LLM回答済み / DB未保存の1 answer
phase	idle / preparing / generating / saving
editing	記憶編集または音声セッション中の排他
presence	表示中クライアントの短いTTL
references	今回の回答に利用したwisdom/persona参照

優先順位

User chat

Memory job

整理中に会話が来た場合、jobをcancelし、未処理rawを残して会話へ譲る。

Saving

Cancel

保存フェーズではcancelを拒否。回答とDBの不整合を避ける。

Direct tool

Gemini

確定的ローカル処理を先に判定し、不要なLLMコスト・待ち時間を削減。

「かぐや自身」の状態を育てる実験レイヤー

OPT-IN / FAIL-OPEN / POSTGRESQL

Kaguya Mindは、ユーザー記憶とは別に「感情」「好み」「未完の話題」を扱う。OFF時はテーブルを触らず、失敗時もコア会話を止めない。



Emotion

happiness / curiosity /
jealousy / concern /
boredom等を時間減衰。時
間帯からenergyを合成。

Trait

かぐや自身の「好き/苦手」
を回答文から保守的に抽
出。否定・仮定・引用を除
外。

表情への反映

Controllerが1分程度でMindのfaceを再評価し、PC/
iPhoneへ同じ mood.changed を配信。Mind OFF時
は簡易Moodへフォールバック。

Open loop

予定・体調など「その後」
が気になる話題を期限付き
で保存。しつこさ防止の
asked回数を管理。

Graph

subject / relation / object /
strengthで「かぐや likes
X」等を保持。

ユーザー記憶と混ぜない

wisdom/personaは「ユーザーについて」。mind_*は「かぐや自身について」。意味の境界をDBレベルでも分離する。

生活感は「軽量・決定的」に作る

frontend/src/living.ts + relationship.py

Living Kaguya / frontend

端末内で、時間帯と利用傾向から行動を選ぶ。

入力	算出
現在時刻	energy（深夜低 / 日中高）
lastSeen	idle / reading / working / playing / snacking / daydreaming / sleeping
hourCounts	よく利用する上位3時間帯をlearnedHoursとして補正
server mood	normal / happy / sleepy / sulky をMind/Controllerから同期

localStorage key: kaguya.life.v1。端末差を減らすため、会話成立時のlast_activityはサーバ由来イベントで更新。

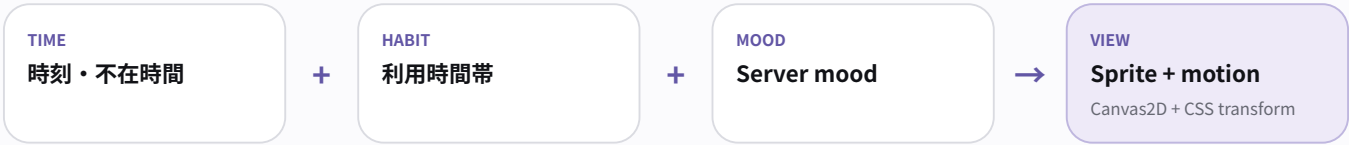
Relationship / backend

関係性はLLMを呼ばず、回数と明示フィードバックで更新。

状態	ルール
慣れ	<5 / <30 / <100 / 100+ chatsで4段階
利用日数	成功会話の日付を最大1000日まで保持
話し方	「短く」「質問減らして」等の明示feedbackを定型hintへ正規化
一時指示	「今回だけ」「今だけ」は長期hintへ入れない

設計意図

「人格を毎回LLMで推論」せず、日常演出は決定的ロジックで出す。コスト・揺らぎ・誤学習を抑える。



raw → wisdom → persona

FACTS → KNOWLEDGE → INTERACTION
STYLE



知恵化

- 未整理のuser発言を1回30件でsnapshot。
- assistant replyは文脈確認にのみ利用し、事実抽出元にはしない。
- 「今回だけ」の指示、挨拶、その場限りの依頼は長期化しない。
- 推測はinferred、明示発言はexplicit。
- 根拠はraw IDのevidenceとして保持。

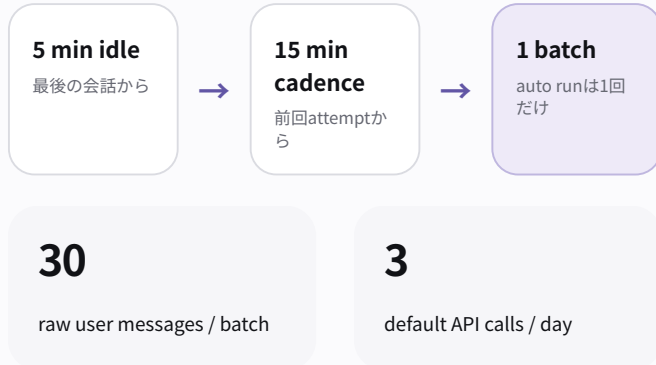
persona更新

- weekly snapshotを作成。
- 別日3日以上 of 明示的根拠があるwisdomのみ候補。
- 1回につきpersona 1項目を調整。
- 固定性格は変更しない。
- previous_value / previous_source_wisdom_ids を保持し、restore可能。

学習は会話を邪魔しない時間に少しずつ

IDLE-FIRST / BOUNDED API CALLS

Automatic trigger



上限は1-10。既定3なら1日最大90件分のrawを処理。上限到達してもrawは保持。

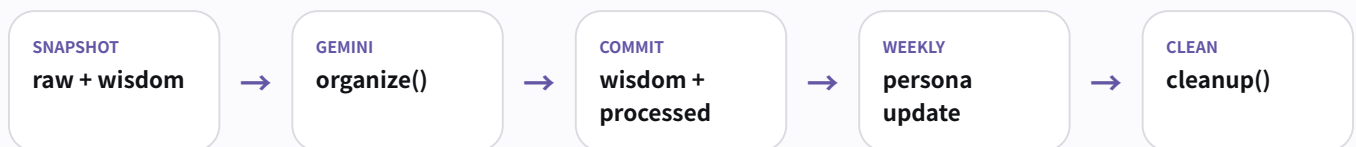
Cancellation semantics

User chat always wins.

整理中に会話が始めると `pause_for_chat()` が job を cancel。snapshot未commit分は未処理として残る。

API予算の記録

外部API呼び出し前にledgerへ予約計上。クラッシュやAPI失敗でも予算消費として扱い、暴走を防ぐ。



手動整理も同じ経路

POST /jobs/run は自動処理と同じJobs.runを使用。別実装にせず、挙動差を作らない。

処理状態と見た目状態を分ける

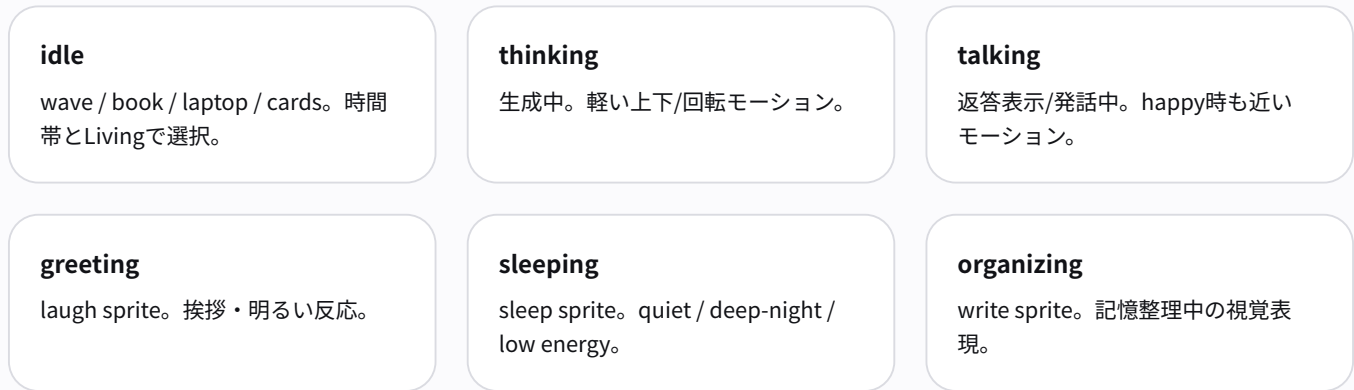
BACKEND STATE ≠ AVATAR STATE

Controller phase

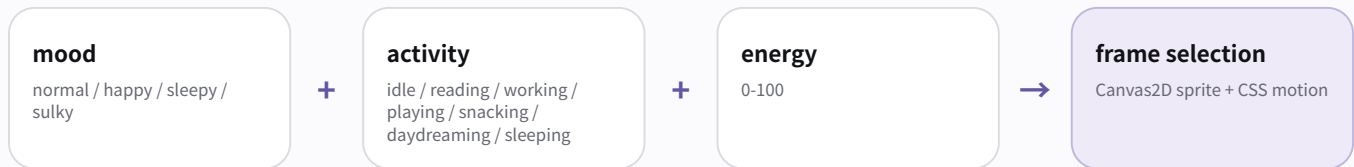


外向け state.changed.state は active中を thinking として表現し、詳細はphaseで示す。

Avatar state



Living mood / activity

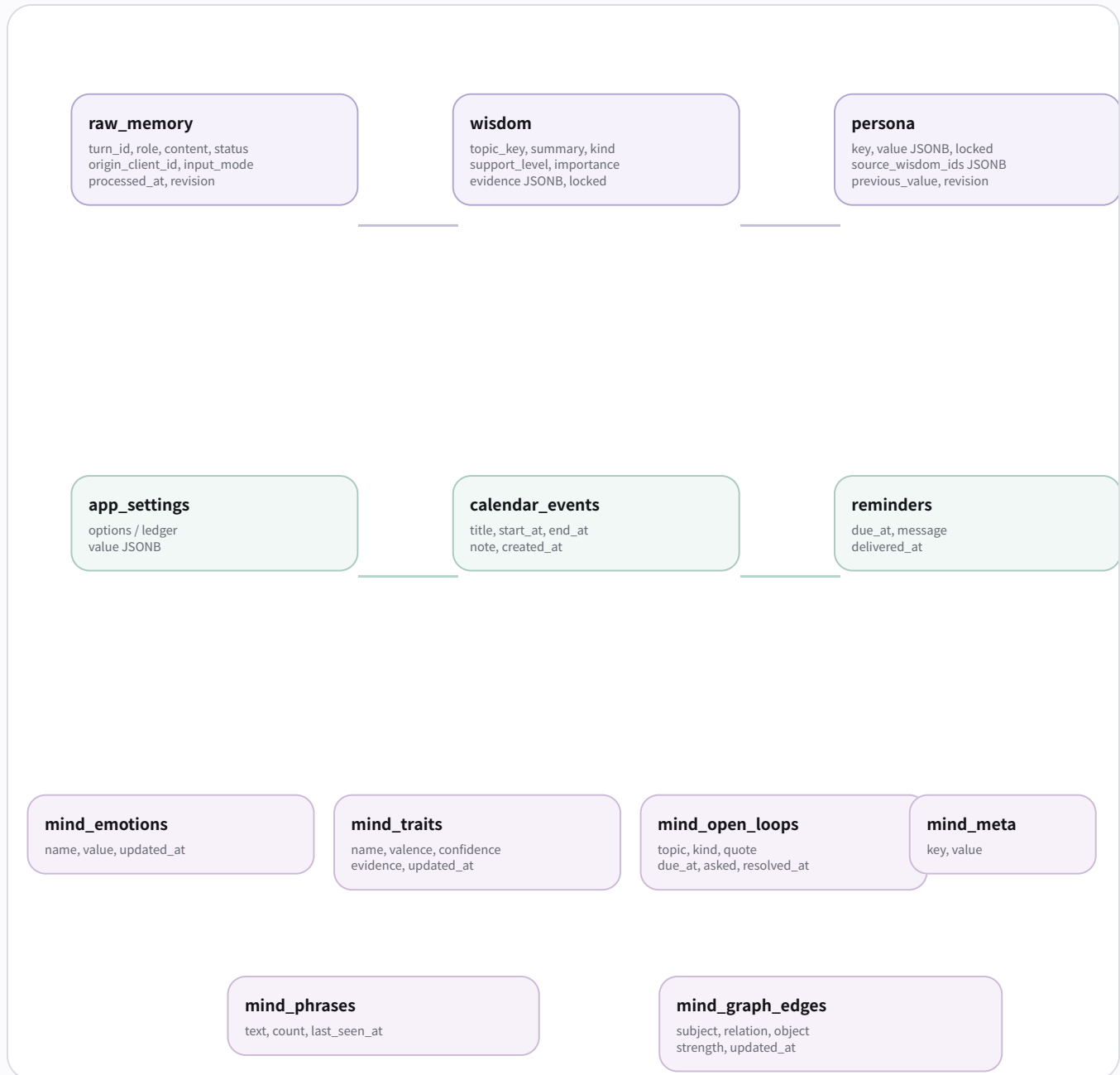


表情のsingle source of truth

MindがONならControllerがサーバ側でmoodを決めて全端末へ配信。端末ごとの判断で顔が食い違わないようにする。

ER図 - PostgreSQLの状態設計

LOGICAL LINKS / JSONB EVIDENCE



● conversation memory ● runtime / utility state ● Kaguya Mind

raw → wisdom → persona は論理参照。wisdom.evidence と persona.source_wisdom_ids はJSONBで根拠IDを保持し、物理FKは張らない。

会話・学習・利用状態テーブル

MIGRATIONS 001-004

Table	主な列	責務 / 制約
raw_memory	id, turn_id, role, content, created_at, processed_at, status, origin_client_id, input_mode, revision, processing_reason	会話原文。 UNIQUE(turn_id, role)。assistantはcompletedのみ。 status = pending/completed/failed/cancelled。
wisdom	topic_key, summary, kind, support_level, importance, evidence, last_seen_at, last_used_at, revision, locked	長期知識。 topic_key unique。evidenceはraw根拠をJSONBで保持。 importance 1-5。
persona	key, value, locked, source_wisdom_ids, revision, previous_value, previous_source_wisdom_ids	相手への接し方。 base_personality / reply_style / addressing / support_style。 restore可能。
app_settings	key, value, updated_at	options と ledger。 UI設定、memory job call budget、relationship countersをJSONBで保持。
calendar_events	id, title, start_at, end_at, note, created_at	ローカル予定。期間 検索用index。 end_at > start_at。
reminders	id, due_at, message, created_at, delivered_at	時刻指定の声かけ。 未配信+期限到来検索用index。ackまでUI表示を保持。

DBアクセス

psycpg 3。会話/予定/設定は共有Database。Kaguya Mindのみ別Database接続を注入し、実験機能障害を分離。

Migration

backend/migrate.py と backend/migrations/*.sql。起動時にローカルmigrationを適用。

Kaguya Mind テーブル

EXPERIMENTAL / PREFIX mind_

Table	主な列	用途
mind_emotions	name, value, updated_at	感情値の現在値。時間減衰の起点をupdated_atで持つ。
mind_traits	name, valence, confidence, evidence, updated_at	かぐや自身の好み/苦手。証拠回数と確信度を持ち、成長表示へ利用。
mind_phrases	text, count, last_seen_at	ユーザーの繰り返し表現。現状は設定表示候補にのみ利用し、応答文体へ直接模倣しない。
mind_graph_edges	subject, relation, object, strength, updated_at	簡易知識グラフ。複合PK(subject, relation, object)。
mind_open_loops	topic, kind, quote, opened_at, due_at, last_asked_at, asked, resolved_at	後で気にかかる予定/体調。due indexで「未解決・期限到来」を抽出。
mind_meta	key, value, updated_at	interaction数など、Mind全体の小さなメタ状態。

OFF

既定OFF

Options.mind_enabled = false。
OFF時はMind tableを読み書きしない。

SAFE

fail-open

Mind例外はログへ記録し、fallbackを返す。通常会話を失敗させない。

RESET

独立リセット

DELETE /mind はmind蓄積のみ削除。会話・wisdom/persona・relationshipは保持。

外向けAPIとWebSocket

FastAPI / SESSION BEARER

Method	Path	用途	主な保護
GET	/health	backend起動確認。instanceを返す。	public
POST	/session	client_idとsession_tokenを払い出す。	max 32 sessions
GET	/history	最新会話。cursor pagination、limit 1-50。	Bearer
GET/PATCH	/settings	options / job status / reminders / mind snapshot。	Bearer + validation
POST	/jobs/run	記憶整理を手動起動。	Bearer + busy guard
GET	/memory-summary	記憶件数・job状態。	Bearer
GET	/memories/{layer}	raw / wisdom / persona / mind の閲覧・検索。	Bearer
GET	/memories/{layer}/{key}/impact	変更影響と確認tokenを取得。	Bearer
PATCH	/memories/{layer}/{key}	訂正/lock/delete。revision+impact_tokenで競合防止。	Bearer + confirmed
POST	/memories/persona/{key}/restore	personaをprevious valueへ戻す。	Bearer + revision
DELETE	/mind	Mind蓄積のリセット。	Bearer + idle
DELETE/POST	/reminders/{id} / ack	予約削除 / 確認済み。	Bearer
WS	/ws	chat.send/cancel/retry_save/presence。状態・progress・completedをbroadcast。	token + Origin
WS	/voice	Gemini Live音声。PCMと字幕、音声byte。	trusted local/Tailscale

WebSocket events

state.changed / chat.accepted / chat.progress /
chat.completed / chat.error / mood.changed /
reminder.due / reminder.ack / jobs.changed /
settings.changed / memories.changed / pc.open

Session boundary

session tokenは現行ではブラウザtab識別とturn
ownership用。セキュリティ境界は
LocalAccessMiddleware / Origin / Tailscale identity側が担
う。

Memory API / PC連携

INTERNAL TOKEN / ALLOWLIST

Internal Memory API / prefix /internal/memory

Path group	用途
/begin, /turns/{id}/complete fail	turn admission / completion / failure
/context, /history, /recall	直近文脈 / 履歴 / 関連記憶
/organize/snapshot commit	raw→wisdomの2段階commit
/weekly/snapshot commit	wisdom→persona
/browse/*	記憶閲覧、impact、変更、restore
/calendar/*, /reminders/*	予定と予約
/references	%LOCALAPPDATA%/KaguyaAI/references 検索

同一FastAPI内でも x-internal-token 付きのHTTP client経由で呼び、外向けsessionでは直接アクセスできない。

PC API / prefix /pc

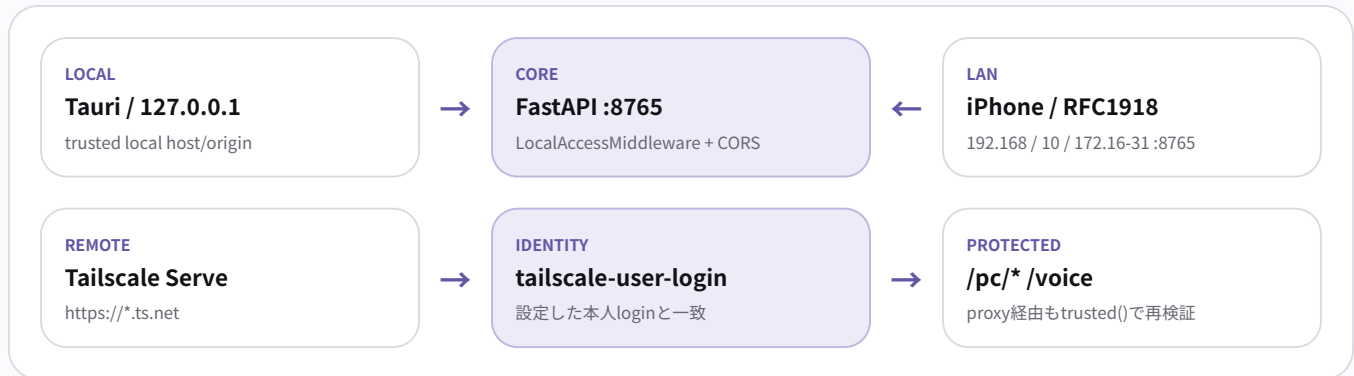
Endpoint	用途
GET /status	登録folder/command、remote URL、voice状態
GET /videos	登録folder内MP4を検索（最大10,000 scan）
POST /videos/ticket	2時間有効のstream ticketを発行
GET HEAD /stream/{token}	ticketでMP4配信
POST /commands/{key}/prepare	BAT内容+hashをsnapshotして確認token
POST /commands/run	confirmed=trueで実行
GET /jobs	自分が開始したBAT job状態

PC操作は allowlist + two-step confirmation

任意パスやクライアント引数は受け付けず、登録済みBATのみ。prepare時SHA-256と設定snapshotを保存し、run時に再照合する。

「本人のPC」境界を崩さない

LOOPBACK / PRIVATE LAN / TAILSCALE



Boundary rules

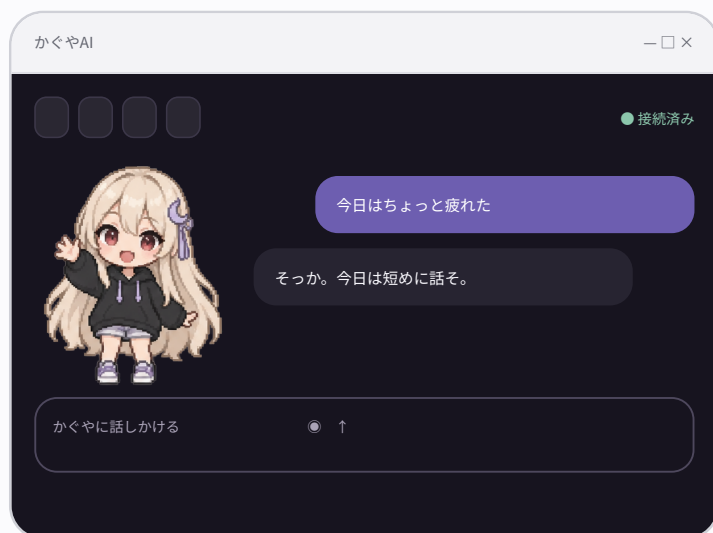
- PC連携設定は backend/pc_access.json の明示allowlist。
- 動画はroot配下の通常MP4のみ。symlink/junctionを辿らない。
- BATは絶対path、.bat、特殊文字制限、client argsなし。
- Uvicornはproxy headerを信用せず、LocalAccessMiddlewareがclient/host/headerを評価。
- .envのAPI key / DB URLはGitへcommitしない。

Origin policy

Origin	許可
Tauri origins	allow list
Vite 127.0.0.1:5173	dev only
RFC1918 :8765	regex
Configured *.ts.net	exact match + login
Public IP / other port	reject

1つのWeb UIをPCとiPhoneで共有

TYPESCRIPT / CSS / CANVAS2D



PC: Tauri window. 通常表示 / 簡易表示、最前面切替、トレイ常駐。

Frontend modules

File	責務
main.ts	session、WebSocket、chat UI、history、memory/settings panel
avatar.ts	Canvas2D sprite、時間帯frame、motion
living.ts	life state、利用時間帯学習、復帰演出
voice.ts	音声capture / /voice WS / PCM playback
pc.ts	PC tab、動画、BAT確認/実行
controls.ts	Tauri/設定等のUI制御
style.css	layout / responsive / PWA safe-area
ui-polish.css	視覚調整レイヤー

Canvas2Dは「アバター描画」に限定

テキストや操作UIはDOM/CSS。アクセシビリティとレスポンス性を壊さず、キャラ演出だけをCanvasへ寄せる。

iPhoneでは会話を最優先

SAFARI / PWA / SAFE AREA



Mobile principles

Header / Character / Composerを固定し、履歴だけをスクロール。

- viewport-fit=cover とsafe-areaを前提に、ホーム画面起動でも操作領域を保つ。
- チャット・記憶・ファイル・設定を上部navで切替。操作数を増やさない。
- 生成中は接続/phaseを短いstatus lineで提示し、本文を遮らない。
- 入力欄は自動拡張しても画面全体を押し出さず、送信/停止/音声の主操作を近接配置。
- 同じfrontend/distをFastAPIが配信し、別iPhoneアプリを作らない。

Apple HIG alignment

情報の重要度を上→下に配置、関連操作をグループ化、余白を一定化。ブランド表現より会話内容を優先し、装飾はキャラクター領域へ限定。

技術スタック

LOCAL-FIRST / BORING TECH WHERE
POSSIBLE

Layer	Technology	役割 / 選定理由
Desktop shell	Tauri 2 / Rust / WebView2	軽量のWindows常駐、透明window、tray、Web UI共有。
Frontend	TypeScript / Vite 6 / CSS	依存を小さく保ち、PC/iPhoneで同一コード。
Avatar	Canvas2D + PNG sprites	128系spriteをシンプルに描画。DOM UIとは分離。
Backend	Python 3.11+ / FastAPI / Uvicorn	HTTP/WS、async I/O、1 workerの明確な状態所有。
DB	PostgreSQL / psycpg 3	会話・長期記憶・設定・Mindを永続化。
Text AI	Google GenAI / GEMINI_MODEL	運用想定: gemini-3.5-flash-lite 。モデルIDは.envで差替可能。
Voice AI	Gemini Live	既定設定: gemini-3.1-flash-live-preview 。音声+字幕。
Local TTS	AivisSpeech / VOICEVOX-compatible	既定: コハク / ノーマル 、localhost:10101。
Remote access	Tailscale Serve	本人のtailnetからHTTPS。Funnel不使用。
Weather	Open-Meteo	API key不要。明示的天気質問はGeminiを経由しない。

FastAPI

WebSocket

PostgreSQL

Tauri 2

TypeScript

Canvas2D

Gemini

AivisSpeech

Tailscale

失敗しても「会話」と「記憶」を壊さない

RECOVERABLE / EXPLICIT / BOUNDED

429

自動再試行しない

rate limitはretryDelayがあれば提示し、ユーザーの手動再試行へ。

5xx

短く1回だけ再試行

500/502/503/504と一部NetworkErrorのみ。長いtimeout後は再試行しない。

DB

pending recovery

起動時にpending user rowをfailedへ戻し、曖昧な生成中状態を残さない。

SAVE

unsaved answer

生成済み回答は保持し、保存のみ再試行。二重課金・回答変化を避ける。

MIND

fail-open

Mindや生活演出は落ちてもコア会話を継続。

TTS

fallback

AivisSpeech unavailableならGemini voiceへ戻し、通話自体は継続。

User-visible feedback

- 接続中 / 回答生成中 / 保存中を明示。
- エラーコードごとに「再試行できるか」を変える。
- DB障害でも設定画面自体は開けるよう一部をbest-effort化。
- 音声エラーは利用中のvoice engine情報を添えて診断可能にする。

Bounded work

- memory jobs: daily call limit。
- voice: 600 sec session cap。
- video scan: 10,000 files。
- BAT timeout: 5-3600 sec、1 job at a time。
- ticket/confirmation/sessionに件数・期限上限。

起動と運用

WINDOWS HOST

Required runtime

Component	用途
Python 3.11+	FastAPI backend
Node.js + npm	frontend build
Rust + MSVC + Windows SDK	Tauri build
WebView2 Runtime	PC UI
PostgreSQL	persistent state
Git	明示update
Tailscale (optional)	remote private HTTPS
AivisSpeech (optional)	local TTS

.env core

```
GEMINI_API_KEY=...
GEMINI_MODEL=gemini-3.5-flash-lite
GEMINI_LIVE_MODEL=gemini-3.1-flash-live-preview
DATABASE_URL=postgresql://kaguya:***@127.0.0.1:5432/
kaguya_ai
```

Start / update

- `start.bat`: migration + local build + launch。通常起動ではGit更新しない。
- `kaguya.bat update`: origin/mainをff-only更新 → build → restart。
- update失敗時も現在版の再起動を試行。logはLocalAppData配下。
- スタートアップ登録はWindows startup folder。AivisSpeechも探索して同時起動可能。

設定はPostgreSQLへ集約

settings.json / calendar.json / mind.db の分散をやめ、app_settings / calendar_events / mind_* へ移行。参照フォルダと.envだけをファイルに残す。

変更したい機能から、触る場所を逆引き

IMPLEMENTATION MAP

やりたいこと	主なファイル	見るポイント
会話処理を変える	controller.py / llm.py / persona.py	admission、tool routing、memory prompt、streaming
記憶ロジックを変える	memory_api.py / memory_store.py / jobs.py	snapshot/commit、recall、cleanup、daily/weekly
Kaguya Mindを変える	mind/engine.py / mind/store.py / tuning.py	emotion decay、trait抽出、open loop、threshold
音声を変える	voice.py / tts.py / frontend/src/voice.ts	Live config、PCM、TTS chunking、fallback
PC操作を増やす	pc.py / pc_access.json / frontend/src/pc.ts	allowlist、prepare→confirm→run、stream ticket
画面を変える	index.html / main.ts / style.css / ui-polish.css	DOM構造、panel、responsive、safe-area
キャラ挙動を変える	avatar.ts / living.ts / mood.py	sprite mapping、time slot、life state、server mood
DB schemaを変える	migrations/*.sql / migrate.py	後方互換、index、runtime storeとの整合
接続境界を変える	config.py / pc.py / main.py	CORS、Origin、trusted proxy、session

変更前に守る不変条件

- 会話は同時に1件。
- 保存失敗で自動再生成しない。
- 未整理rawを失わない。
- Mind/演出障害で会話を止めない。
- PC操作はallowlist + user confirmation。

設計レビュー観点

- Gemini呼び出しを本当に増やす必要があるか。
- 端末ごとに状態を持って食い違わないか。
- DB commit前後で再送されてもidempotentか。
- ネットワーク障害後に復旧可能か。
- UIに技術用語を露出しすぎていないか。